Difference Between final, finally, and finalize in Java

1. Introduction

In Java, `final`, `finally`, and `finalize` are three different concepts that often confuse beginners. They serve distinct purposes in the language:

- **`final`** is a keyword used for **variables, methods, and classes** to impose restrictions.
- **`finally`** is a block used in **exception handling** to execute crucial code regardless of whether an exception occurs.
- **`finalize`** is a method in **Garbage Collection (GC)** that is called before an object is destroyed.

Let's explore each of them in detail with examples.

---

2. `final` Keyword

The `final` keyword can be applied to **variables, methods, and classes** to enforce restrictions.

2.1 `final` with Variables

A `final` variable becomes a constant, meaning its value cannot be changed once assigned.

Example:

```
class ExampleFinalVariable {
    final int speedLimit = 100; // Final variable

    void changeSpeed() {
        // speedLimit = 120; // Error: Cannot assign a value to final
variable
        System.out.println("Speed Limit: " + speedLimit);
    }

    public static void main(String[] args) {
        ExampleFinalVariable obj = new ExampleFinalVariable();
        obj.changeSpeed();
    }
}
```

2.2 `final` with Methods

A method declared as `final` **cannot be overridden** in a subclass.

Example:

```
class Parent {
    final void show() {
        System.out.println("This is a final method.");
    }
}

class Child extends Parent {
    // void show() { // Error: Cannot override the final method from Parent
    //     System.out.println("Trying to override.");
    // }
}
```

2.3 `final` with Classes

A `final` class **cannot be extended (inherited)**.

Example:

```
final class FinalClass {
    void display() {
        System.out.println("This is a final class.");
    }
}

// class SubClass extends FinalClass { // Error: Cannot inherit from final
class
// }
```

---

3. `finally` Block

The `finally` block is used in exception handling and is **always executed** whether an exception occurs or not. It is mainly used for **resource cleanup** (e.g., closing files, database connections).

Example:

```
class FinallyExample {
    public static void main(String[] args) {
        try {
            int data = 10 / 0; // This will cause an exception
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e);
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Exception caught: java.lang.ArithmeticException: / by zero
Finally block executed.
```

Even if an exception is not thrown, the `finally` block executes:

```
class FinallyExampleWithoutException {
    public static void main(String[] args) {
        try {
            int data = 10 / 2; // No exception
            System.out.println("Result: " + data);
        } catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e);
        } finally {
            System.out.println("Finally block executed.");
        }
    }
}
```

Output:

```
Result: 5
Finally block executed.
```

---

4. `finalize()` Method

The `finalize()` method is called by the **Garbage Collector (GC)** before an object is removed from memory. It is used for **cleaning up resources**, but its use is discouraged in favor of `try-with-resources` or explicit cleanup.

Example:

```
class FinalizeExample {
    @Override
    protected void finalize() throws Throwable {
        System.out.println("Finalize method called before garbage
collection.");
    }

    public static void main(String[] args) {
        FinalizeExample obj = new FinalizeExample();
        obj = null; // Making object eligible for garbage collection
        System.gc(); // Requesting Garbage Collection
        System.out.println("Main method execution completed.");
    }
}
```

Output (May Vary):

```
Main method execution completed.
Finalize method called before garbage collection.
```

Note:

- The execution of `finalize()` is **not guaranteed** because garbage collection occurs **at the discretion** of the JVM.
- From **Java 9**, `finalize()` is **deprecated** due to performance issues.

---

5. Key Differences

Feature	`final`	`finally`	`finalize()`
Purpose	Restricts variables, methods, and classes	Ensures execution of critical code (exception handling)	Called by Garbage Collector before object deletion
Usage	Used with variables, methods, and classes	Used in exception handling	Defined in `Object` class
Execution	Enforced by compiler	Always executed	JVM calls it before garbage collection
Overridable	No	Not applicable	Yes (can override)
When it runs	Compile-time	Run-time	Before object is garbage collected

---

6. Conclusion

- Use `final` when you **want to prevent modification or inheritance**.
- Use `finally` to **ensure resource cleanup** in exception handling.
- Avoid using `finalize()`, as **it is unpredictable and deprecated**.

Best Practices:

- Prefer **try-with-resources** (`AutoCloseable`) instead of relying on `finalize()`.
- Use `finally` **for critical code execution** (like closing connections).
- Use `final` **to enforce immutability** or prevent unintended method overriding.

Would you like an advanced topic on **Garbage Collection strategies** in Java? 🚀